西北大学

第一章：Python 语言入门

谭振宇

西北大学城市与环境学院

2026 年 05 月 06 日

Python 语言闲话

基本数据类型及运算符

字符串及输入输出

序列与字典

流程控制

函数与模块

文件操作

面向对象编程与函数式编程

本章小结

什么是 Python?

Python 是一种**解释型**的通用**高级动态编程语言**。

解释型语言

Python 编译器将源代码(.py)编译为字节码(.pyc)，由 Python 解释器解释执行。跨平台性良好。

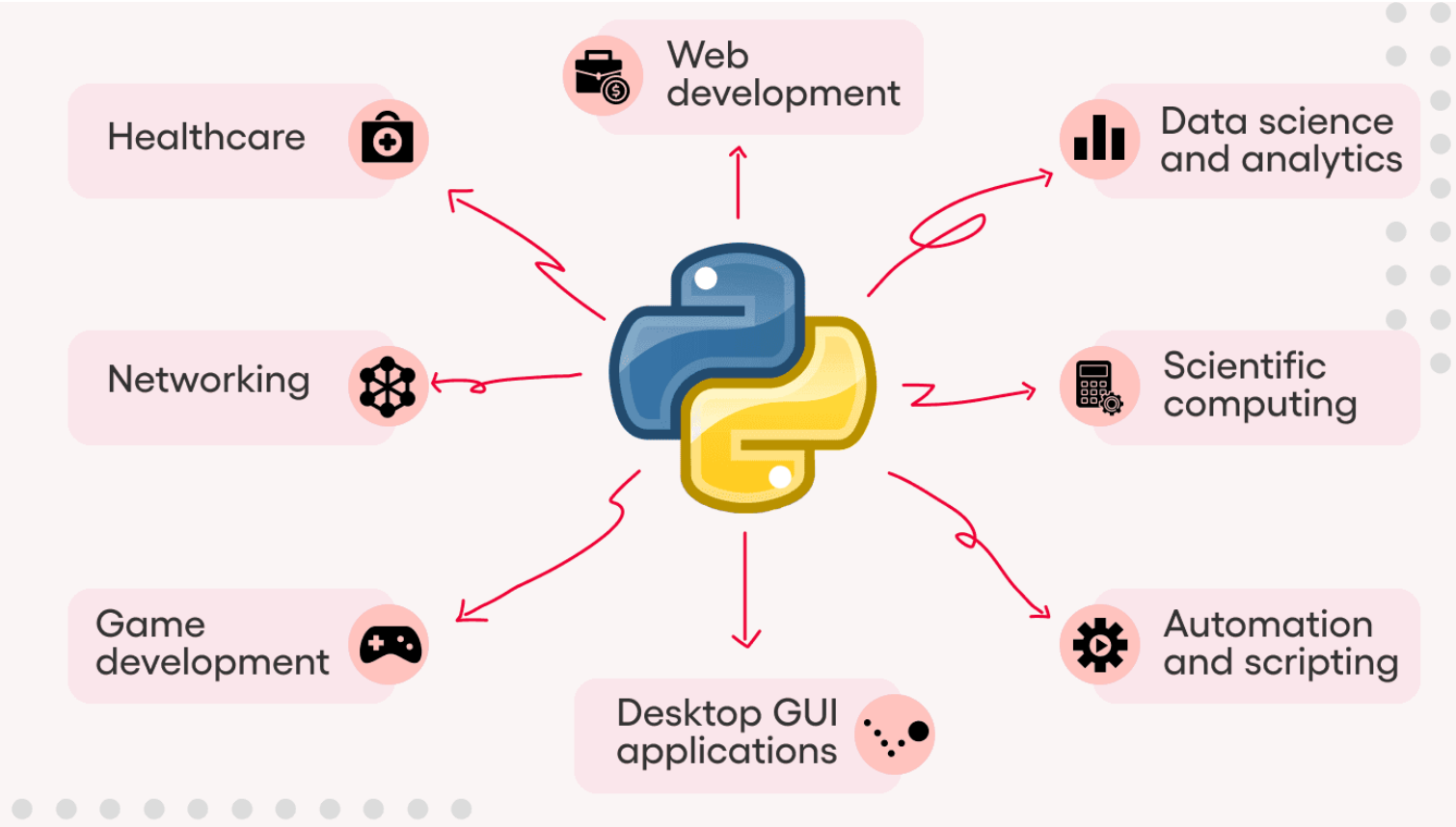
动态编程语言

在程序运行过程中能够修改自身程序结构。变量类型在运行过程中动态确定，开发效率高。

通用编程语言

被称为“胶水语言”，可用于系统运维、Web 开发、GUI 桌面程序、数据科学、人工智能等领域。

什么是 Python?


Web development
Healthcare
Data science and analytics
Networking
Scientific computing
Game development
Automation and scripting
Desktop GUI applications

Python 的优缺点及学习方法

西北大学

优点

- 开源免费，简单易学
- 海量优秀的第三方库，投入产出比极高

缺点

- 运行速度慢（解释型语言）
- 程序维护相对困难（动态类型语言）

学习方法

推荐**“先整体，后局部，再整体”**的学习方法。首先对全局有了解，其次深挖所需部分，最后通过实践形成自己的理解。

Python 与 Conda 简介

对于初学者而言，构建合适的开发环境至关重要：

Python 编译器环境

- 官网直接下载的基础环境
- 内置 `pip` 安装第三方库
- 带有基本的 IDLE 交互界面

Conda 包管理工具

- 卓越的包（库）依赖管理
- 轻松构建、隔离不同的 Python 虚拟环境
- 推荐使用 Miniconda 或 Anaconda

Shell 与 终端 (Terminal) 环境

命令行是进行 Python 高级管理的基础，区分 Shell 与 Terminal：

- **Shell (命令行解释器)**：操作系统提供的人机交互接口。如 Windows 的 `CMD` 与 `PowerShell`，macOS/Linux 的 `Bash` 与 `Zsh`。
- **Terminal (终端，前端交互)**：与用户交互的软件界面。

推荐组合：在 Windows 系统中，推荐使用强大的 `Windows Terminal` 搭载 `PowerShell` 进行开发管理，支持多标签和自定义配置。Mac 系统推荐使用自带的 Terminal 或 `iTerm2`。

Conda 环境管理常见命令

使用 Conda 管理虚拟环境与第三方库，可以有效避免版本冲突：

```
# 1. 创建名为 osgeo 的虚拟环境，指定 Python 3.7
conda create --name osgeo python=3.7

# 2. 激活虚拟环境（退出使用 conda deactivate）
conda activate osgeo

# 3. 在当前环境中安装 numpy 科学计算库
conda install --channel conda-forge numpy

# 也可以补充使用 pip 安装一些 Conda 下没有的库
pip install numpy
```

其他常用操作：`conda env list`（查看所有环境），`conda list`（查看已安装包）。

命令行交互与 Jupyter Notebook

IPython 增强型终端

- 支持 Tab 代码提示与自动缩进
- 内置魔术命令（如 `%ls`, `%cd`）
- 通过追加 `?` 或 `??` 快速获取函数帮助文档或源代码

Jupyter Notebook / Lab

- 数据科学家首选，网页交互式编程
- 支持 Markdown 笔记、LaTeX 公式
- 代码与图表混合、实时呈现数据可视化，适合实验探索与分析

集成开发环境 (IDE)：PyCharm

IDE 将编辑器、环境管理、调试器等工具有机结合，大幅提升工程开发效率。

强大的工程开发支持

- **项目与环境管理**：能自动识别并关联 Conda 创建的虚拟环境。
- **深度调试 (Debug)**：支持可视化打断点（Breakpoints）、逐行与跳出（Step Over / Step Into）、控制台交互查看变量（Console）。
- **代码提示与规范**：内置 PEP 8 风格指南检查，变量命名校验（小写加下划线 `snake_case` 和帕斯卡命名 `PascalCase` 等），一键重构代码格式功能。

随着 AI 技术的发展，VSCode 编辑器被广泛应用（Positron，Antigravity 等）

Python 语言闲话

基本数据类型及运算符

字符串及输入输出

序列与字典

流程控制

函数与模块

文件操作

面向对象编程与函数式编程

本章小结

Python 基本数据类型

数据结构描述了数据在计算机中的存储和组织方式。

- **Number（数字）**：分为整型(`int`)、浮点型(`float`)以及复数类型(`complex`)。
- **String（字符串）**：单引号或双引号引起来的字符序列。
- **Boolean（布尔型）**：取值仅为 `True` 和 `False`。
- **None（空值）**：表示该对象不存在。
- **Byte（二进制字节）**：以 `b` 开头，如 `b'hello'`。

基本数据类型示例

Number: `10, 3.14, 1+2j`

String: `"Hello", 'NWU'`

Boolean: `True, False`

None: `None`

Byte: `b'hello'`

Python 基本数据类型

高级的数据结构包括：

- **Tuple（元组）**：不可变的有序数据集合，使用小括号 ()。
- **List（列表）**：可变的有序数据集合，使用方括号 []。
- **Dictionary（字典）**：键值对集合，使用花括号 {}。键必须不可变。
- **Set（集合）**：无序、不重复的元素集合，使用花括号 {}。

高级数据结构示例

Tuple: `(1, 2, 'a')`

List: `[1, 2, 'a']`

Dict: `{'k': 'v'}`

Set: `{1, 2, 3}`

Python 基本数据类型

数字、字符串、布尔型、元组属于**不可变类型**；列表、字典、集合属于**可变类型**。

可变与不可变类型示例

```python
t = (1, 2)
# t[0] = 3   # TypeError: 不支持修改

s = "NWU"
# s[0] = "n" # TypeError: 不支持修改
```

```python
lst = [1, 2]
lst[0] = 3   # 成功，lst变为 [3, 2]

d = {'k': 'v'}
d['k'] = 'x' # 成功，d变为 {'k': 'x'}
```

数学运算符

Python 支持基础的数学运算：

```
1 + 1.0      # 加法，输出 2.0
2.5 - 0.0    # 减法，输出 2.5
2 * 3        # 乘法，输出 6
1 / 2        # 除法，输出 0.5
1 // 2       # 整除（向下取整），输出 0
1 % 2        # 取模（余数），输出 1
2 ** 3       # 乘方运算，输出 8
```

比较与赋值运算符

比较运算符用于大小判断，返回布尔值：$=, \neq, >, <, \geq, \leq$。

赋值运算符 (=) 将右侧值赋给左侧变量。支持与算数运算符结合：

```
x = 10              # 赋值
x, y = 10, 0.5      # 变量的多重赋值
x, y = y, x         # 交换x和y的值
x += 2              # 等价于 x = x + 2
```

逻辑运算符

处理多个布尔值关系的运算符：and (与)、or (或)、not (非)。

```
x, y = False, True
x and y     # 返回 False
x or y      # 返回 True
not x       # 返回 True

# 短路逻辑
10 and 20   # 因为 10 为真，返回后面的 20
10 or 20    # 因为 10 为真，返回 10，不再判定后面的 20
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

字符串 (String) 操作

字符串是不可变序列，支持切片、索引和运算：

```python
a = 'Hello '
b = "world!"
a + b          # 拼接运算: 'Hello world!'
a * 2          # 重复运算: 'Hello Hello '
a[0]           # 索引首字符: 'H'
a[-1]          # 索引末字符 (倒数第一个): ' '
a[0:3]         # 简单切片: 'Hel'
'e' in a       # 成员检查: True
```

字符串的高级用法：切片与 f-String

西北大学

切片操作 (Slice)

语法为 [start:stop:step]。起始索引默认 0，终止默认 −1，步长默认 1。

```
a[1:-1:2]  # 获取索引 1 到 -1（不含）之间，步长为 2 的字符序列
```

f-string 格式化

以字母 f 开头，将变量包含在花括号 {} 中，方便拼接和格式化。

```
x = 3.1415926
f'PI is {x:.3}'  # 控制有效数字，输出 'PI is 3.14'
```

终端输入与输出

利用 input() 获取输入，print() 打印输出。

```python
age = input('Please input your age: ')
print(f'I am {age} years old.')

# 注意: input() 函数默认将所有输入处理为字符串 (str)。
type(age)  # 输出 <class 'str'>
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

列表 (List) 基础概念

列表是 Python 最常用的容器之一，支持存储不同类型数据并可随时修改。

```
colls = [1, 'good', 2.0, print]
print(colls[0], colls[-1])      # 访问第一个和最后一个元素
print(colls[2:])                # 切片访问从第三个到末尾的所有元素
```

列表推导式 (List Comprehension)

列表推导式是极具 Python 特色的语法，用于优雅地生成列表：

```python
nums = [i for i in range(5)]
# [0, 1, 2, 3, 4]

squares = [i ** 2 for i in range(5)]
# [0, 1, 4, 9, 16]

# 结合if进行过滤
evens = [i for i in nums if i % 2 == 0]
# [0, 2, 4]
```

列表的增删改操作

西北大学

掌握对列表元素的动态调整：

```
nums = [0, 1, 2, 3]

nums.append(5)          # 尾部添加，变为 [0, 1, 2, 3, 5]
nums.insert(0, 100)     # 指定位置插入，变为 [100, 0, 1, 2, 3, 5]

del nums[0]             # 关键字删除，变为 [0, 1, 2, 3, 5]

nums[0] = -1            # 赋值修改，变为 [-1, 1, 2, 3, 5]
```

列表的常用内置函数

Python 对列表有丰富的内置支持：

```
1  len(nums)                    # 获取列表长度（元素个数）
2  max(nums) / min(nums)        # 求最值
3
4  sorted(nums, reverse=True)   # 返回新的降序列表，不改变原列表
5  nums.sort()                  # 就地排序，改变原列表
6
7  9 in nums                    # 判断元素是否在列表中
8  nums + [10, 11]              # 列表拼接
```

元组 (Tuple) 结构

元组与列表类似，但**不可变**，一旦创建无法修改内部元素。

```python
hello = ('你好', '周杰伦')
hello = tuple(['你好', '周杰伦']) # 从列表转为元组

# 元组同样支持索引与切片
hello[0]                    # '你好'
nums = (0, 1, 2, 3, 4)
nums[1:-1:2]                # (1, 3)
```

为何使用元组?

元组的不可变性保障了数据的安全性，多用于函数的多返回值或固定配置信息的存储。

字典 (Dictionary) 结构

字典存储**键值对 (Key-Value)**，通过不可变的键来快速获取对应的值。

```
dates = {'星期天': 'Sunday', '星期一': 'Monday'}

len(dates)                    # 返回键值对个数
dates['星期天']               # 获取键对应的值: 'Sunday'

dates['星期六'] = 'Sat'     # 新增或修改键值对
del dates['星期六']          # 删除键值对
```

字典的遍历操作

通过内置方法，我们可以灵活地遍历字典的键或值：

```python
# 遍历所有的键
for key in dates:
    print(key, dates[key])

# 同时遍历键和值
for key, value in dates.items():
    print(f'{key} → {value}')
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

Python 编码规范

良好的编码风格有助于代码维护与协作：

- 使用 **4 个空格** 进行缩进，不可与 Tab 混用。
- 使用 # 进行单行注释，""" 进行多行注释。
- 变量和函数使用**蛇形命名法** (`snake_case`)。
- 类名使用**帕斯卡命名法** (`PascalCase`)。
- 每个顶级函数或类之间建议空两行，提高可读性。

条件结构 (if-elif-else)

通过条件分支实现程序的逻辑控制：

```
if 1 > 2:
    print('1大于2')
elif 1 < 2:
    print('1小于2')
else:
    print('1等于2')
```

除了 `if-else`，进阶语法允许条件嵌套，或利用 `in` 与 `isinstance` 做高级判断。

循环结构：while 循环

while 循环在条件为 True 时持续执行，也支持与 else 结合（条件变假且正常退出时执行）。

```
i, result = 1, 0
while i < 100:
    result += i
    i += 1
else:
    print(f"Loop finished, result={result}")
```

循环结构：for-in 循环

主要用于遍历可迭代对象（列表、元组、字符串等）或使用 range() 构造序列。

```python
for i in range(5):
    print(i, end=' ')
# 输出: 0 1 2 3 4

# range(start, stop, step)
for i in range(3, 10, 2):
    print(i, end=' ')
# 输出: 3 5 7 9
```

循环控制：break 与 continue

西北大学

在满足特定条件时主动控制并改变循环流向。

- continue：跳过本次循环的剩余语句，直接进入下一次循环。
- break：完全终止当前所在的循环结构。

```
for i in range(1, 10):
    if i in (4, 7):
        continue    # 跳过 4 和 7
    if i == 9:
        break       # 提前结束循环
    print(i)
```

迭代器 (Iterator)

迭代器是按需提供数据的对象，利用 iter() 和 next() 函数进行遍历。

```
greeting = 'How'
it = iter(greeting)

print(next(it))  # 'H'
print(next(it))  # 'o'
print(next(it))  # 'w'
print(next(it))  # 触发 StopIteration 异常
```

事实上 for-in 循环的底层每次也会调用 next() 机制。

生成器 (Generator)

使用了 yield 关键字的函数即为生成器。每次 yield 都会产生一个值并将函数暂停，省去大量内存占用。

```python
def fibonacci(n):
    a, b, count = 0, 1, 0
    while count < n:
        yield a
        a, b = b, a + b
        count += 1

for val in fibonacci(5):
    print(val) # 0, 1, 1, 2, 3
```

异常处理

处理程序运行中不可预见的错误，提高软件健壮性：

```python
try:
    # 尝试执行可能发生异常的代码
    y = 2 / 0
except ZeroDivisionError as e:
    # 发生异常时执行的代码
    print(f"Error: {e}")
else:
    # 未发生异常时执行
    print("No errors.")
finally:
    # 无论是否发生异常必将执行
    print("Always executed.")
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

函数的定义与调用

函数是最小的功能代码单元，可提高代码的复用率和可读性。

```python
def add(x, y=1):
    """文档字符串：计算x与y之和"""
    return x + y

add(10)        # 使用默认值 y=1，返回 11
add(10, 20)    # 覆盖默认值，返回 30
```

深入理解参数传递

Python 的变量传递视**对象是否可变**而定：

- 传递**不可变对象**（数字、元组）：等同于值传递，函数内修改不影响原变量。
- 传递**可变对象**（列表、字典）：等同于引用传递，直接修改原数据。

```python
def modify(a, b):
    a += b

x, y = 1, 2
modify(x, y) # a和b不可变，x=1 不发生改变

lst1, lst2 = [1], [2]
modify(lst1, lst2) # 发生改变，此时 lst1=[1, 2]
```

西北大学

不定长参数与匿名函数

- `*args` 会将额外位置参数组成元组。
- `**kwargs` 会将额外的关键字参数组成字典。

Lambda 匿名函数

`lambda 参数: 表达式`。多用于高阶函数的极简传参，无需定义完整函数。

```
func = lambda x, y: x + y
func(1, 2) # 返回 3
```

模块与包的概念

- **模块 (Module)**：一个 .py 文件，可定义函数、类和变量。
- **包 (Package)**：包含多个模块的目录，内含 __init__.py 文件，以支持分层导入。
- **库 (Library)**：多个包的集合，提供了特定主题相关的功能。

```python
import os
from pathlib import Path

# 使用模块中的方法
os.path.isdir('.')
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

跨平台文件路径 (Pathlib)

做空间数据批处理必不可少的路径操作对象：pathlib.Path。

```python
from pathlib import Path

p = Path() / 'example.txt'    # 利用斜杠进行路径拼接
p.exists()                    # 判断是否存在
p.is_file()                   # 判断是否为文件
p.resolve()                   # 解析为绝对路径
p.parent                      # 获取直系父目录
p.name                        # 获取带后缀的文件全名
p.stem                        # 获取文件主名（不含后缀）
```

路径的常用操作 (shutil)

结合内置 shutil 模块进行复杂文件或目录的移动操作：

```python
import shutil
from pathlib import Path

d = Path('Music')
d.mkdir() # 创建目录

shutil.copy('src.txt', 'dst.txt')          # 文件复制
shutil.move('old.txt', 'new.txt')          # 文件移动或重命名
shutil.copytree('src_dir', 'dst_dir')      # 文件夹完全复制
shutil.rmtree('dst_dir')                   # 文件夹整个移除
Path('file.txt').unlink()                  # 使用Path对象进行文件删除
```

读取文本文件

通常采用 with open() 使得文件在读写结束后自动安全关闭。

```python
with open('青花瓷.txt', mode='r', encoding='utf-8') as f:
    for line in f.readlines():
        print(line.strip())

# mode 表:
# 'r' 只读(默认)   'w' 覆盖写入   'a' 追加写入
# 'b' 二进制模式   't' 文本模式(默认)
```

读取二进制与文件批处理

底层处理空间数据时（如 Shapefile），往往需要通过 struct 解析二进制文件：

```python
import struct
with open('data.shp', 'rb') as f:
    header = f.read(100)
    # 按特定数据格式大端无符号整型解包
    file_code = struct.unpack(">i", header[:4])[0]
```

数据批处理示例：

```python
# 递归找到所有 .py 结尾文件
for f in Path('.').glob('**/*.py'):
    print(f)
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

面向对象：类和对象

面向对象编程把对象作为程序的基本单元。类是一组相关数据(**属性**)和操作函数的封装(**方法**)。

```python
class Rectangle(object):
    def __init__(self, w, h):
        self.width = w
        self.height = h

    def area(self):
        return self.width * self.height

rect = Rectangle(10, 20)
print(rect.area())
```

继承与多态

子类可以继承父类复用代码，并能重写（Overwrite）父类的方法，形成多态行为。

```
class Shape(object):
    def area(self):
      pass

class Circle(Shape):
    def __init__(self, r):
      self.r = r
    def area(self):
      return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h):
      self.w, self.h = w, h
```

继承与多态

```
14      def area(self):
15          return self.w * self.h
16
17  # 多态：同一接口调用，产生不同的行为
18  shapes = [Circle(5), Rectangle(3, 4)]
19  for s in shapes:
20      # 不需关心 s 的具体类型，只需调用 area 即可
21      print(f"{type(s).__name__} 面积为: {s.area()}")
```

函数式编程

关注处理的结果，基于纯函数与数据的流式组装映射。 核心函数：map(), reduce(), filter()。

```
# map: 对数据序列作一对一映射
ls = list(map(lambda x: x*2, [1, 2, 3])) # 返回 [2,4,6]

# filter: 条件过滤序列数据
ls = list(filter(lambda x: x>1, [1, 2, 3])) # 返回 [2,3]

# reduce: 累积归并输出单一值
from functools import reduce
res = reduce(lambda x,y: x+y, [1, 2, 3]) # 返回 6
```

Python 语言闲话
基本数据类型及运算符
字符串及输入输出
序列与字典
流程控制
函数与模块
文件操作
面向对象编程与函数式编程
本章小结

本章知识要点回顾

核心内容

- **数据类型与运算**：深入掌握字符串、列表、字典等容器。
- **流程控制**：熟练使用分支、循环及异常处理构建程序逻辑。
- **函数与模块**：理解参数传递，提高代码模块化复用。
- **文件与批处理**：灵活运用 pathlib 针对数据做批次处理。
- **编程思想**：领会面向对象与函数式编程在开发中的优势。

随堂练习

1. 制作一个简易的计算器，能够实现常见的数学运算。
2. 在不依赖任何第三方库的情况下，制作一个简易的日历。用户通过输入年份和月份，可以打印出该月份的日历表。
3. 自定义一个类 `Matrix`，并实现矩阵的加法和乘法运算。